

KemdiCode MCP v3: Lorenz-Inspired Context Compaction for AI-Assisted Software Engineering

Dawid Irzyk, M.Sc. Eng.

Abstract—This paper presents KemdiCode MCP Server v3.0.0, an open-source Model Context Protocol (MCP) server providing 107 specialized tools for AI cognition, multi-agent coordination, persistent memory, and distributed LLM orchestration. Version 3.0 introduces *Lorenz-inspired context compaction*—three algorithms drawn from chaos theory and dynamical systems for intelligent context window management: Poincaré section phase detection using Jensen-Shannon divergence to identify topic transitions, Lorenz attractor orbit compression using TF-IDF cosine similarity to detect and collapse repeating reasoning patterns, and perturbation impact analysis measuring each item’s contribution to the information landscape. Additionally, 39 tools duplicating native IDE AI capabilities (file operations, git, line editing, code review) have been removed, refocusing the server on capabilities that AI IDEs cannot provide natively. The three-layer distributed bus architecture (L₃ ClusterBus, L₂ DataFlowBus, L₁ GlobalEventBus) and the eight-store cognition subsystem are retained and extended. Validation confirms 694 passing tests and compatibility with four major MCP clients.

Index Terms—Model Context Protocol, context compaction, Lorenz attractor, chaos theory, large language models, multi-agent systems, distributed systems, AI-assisted development

I. INTRODUCTION

AI-ASSISTED development tools face a fundamental tension: as reasoning chains grow longer, the fixed context window forces lossy compression. Which thoughts to keep? Which to discard? This is the *context compaction problem*—and it maps directly to sensitivity to initial conditions in chaotic dynamical systems.

In Lorenz’s seminal meteorological model [5], infinitesimal perturbations to initial conditions produce exponentially divergent trajectories. Context compaction is the inverse problem: given a trajectory (thinking chain), find the minimal perturbation (compression) that preserves the system’s qualitative behavior. If we remove the wrong thought, the AI’s subsequent reasoning diverges catastrophically—the butterfly effect applied to cognition.

KemdiCode MCP Server v3.0.0 addresses this through three algorithms inspired by chaos theory and dynamical systems:

- (i) *Phase detection via Poincaré sections*: Identify topic transitions in thinking chains using Jensen-Shannon divergence [8]. Phase boundaries carry maximum information and must be preserved during compaction.
- (ii) *Orbit compression*: Detect repeating patterns in AI reasoning—analogue to orbits around the Lorenz attrac-

tor’s “eyes”—using TF-IDF cosine similarity matrices, and collapse duplicate cycles.

- (iii) *Perturbation impact analysis*: Measure each item’s contribution to the information landscape via JSD of the full context versus the context-without-item. Items with high perturbation impact are causal anchors.

Concurrently, v3.0 removes 39 tools that duplicate capabilities provided natively by modern AI IDEs (Claude Code, Cursor, KiroCode, RooCode). File operations, git commands, line editing, code review, and project management tools have been excised, leaving 107 tools focused on cognition, multi-agent coordination, persistent memory, kanban task management, cluster bus with LLM magistrale, and structured output—capabilities that IDEs *cannot* provide natively.

The remainder of this paper is organized as follows. Section II surveys related work. Section III presents the Lorenz-inspired context compaction framework. Section IV describes the system architecture. Section V details the three protocol layers. Section VI covers the LLM Magistrale. Section VII describes the cognition subsystem. Section VIII catalogs the tool categories. Section IX addresses multi-agent coordination. Section X discusses security. Section XII presents the evaluation. Section XIII concludes.

II. RELATED WORK

The Model Context Protocol builds on the architectural patterns established by the Language Server Protocol (LSP) [4], which standardized language intelligence features across IDEs. While LSP focuses on language-specific operations (completion, diagnostics, refactoring), MCP generalizes the client-server pattern to arbitrary tool invocations for AI assistants.

Multi-agent LLM frameworks have been explored extensively. AutoGen [16] enables multi-agent conversation through a conversable agent abstraction. MetaGPT [17] introduces role-based agent coordination. Li et al. [14] survey workflow patterns and infrastructure requirements. Guo et al. [15] provide a comprehensive taxonomy of multi-agent architectures. KemdiCode differs from these frameworks by operating at the protocol level—it exposes coordination primitives as MCP tools rather than implementing a fixed agent architecture.

Context window management has received increasing attention as LLMs tackle multi-step reasoning tasks. Recursive summarization [18] compresses conversation history but loses structural information. Retrieval-augmented generation [19] sidesteps the problem by keeping a vector store, but cannot preserve causal reasoning chains. Our approach treats the thinking chain as a dynamical trajectory and applies tools

D. Irzyk, M.Sc. Eng., is with Kemdi Sp. z o.o., Poland (e-mail: dawid@kemdi.pl).

This work is licensed under the GNU General Public License v3.0. Source code is available at <https://git.kemdi.pl>.

from chaos theory to identify which elements are structurally essential.

For distributed messaging, Redis Pub/Sub [9] provides the foundation for KemdiCode’s inter-cluster communication. Tree-sitter [11] provides the AST infrastructure. Secret sharing follows Shamir’s original construction [10].

III. LORENZ-INSPIRED CONTEXT COMPACTION

The central insight of v3.0 is that *context compaction is a sensitivity-to-initial-conditions problem*. The Lorenz system [5]:

$$\dot{x} = \sigma(y - x), \quad \dot{y} = x(\rho - z) - y, \quad \dot{z} = xy - \beta z \quad (1)$$

exhibits three properties that map directly to the context compaction problem:

- 1) **Sensitivity to initial conditions:** Removing a critical thought (perturbing the initial state) causes the AI’s subsequent reasoning to diverge exponentially.
- 2) **Strange attractor structure:** AI reasoning tends to orbit around a finite set of conceptual “eyes” (key ideas), tracing similar but not identical paths—creating compressible redundancy.
- 3) **Phase transitions:** The Lorenz system exhibits rapid transitions between the two attractor lobes, analogous to topic switches in thinking chains.

We formalize three algorithms that exploit these properties.

A. Phase Detection via Poincaré Sections

In dynamical systems, a Poincaré section is a hyperplane transverse to the flow, used to reduce continuous dynamics to a discrete map [6]. We apply this concept to thinking chains: the “flow” is the sequence of thoughts, and “Poincaré sections” are the points where the topic changes direction.

Definition 1 (Phase Boundary): Given a thinking chain $\mathcal{T} = (t_1, t_2, \dots, t_n)$ where each t_i has content c_i , a *phase boundary* exists between t_i and t_{i+1} if:

$$\text{JSD}(P(c_i) \| P(c_{i+1})) > \tau_{\text{phase}} \quad (2)$$

where $P(c)$ is the token probability distribution of content c and $\tau_{\text{phase}} = 0.35$ is the phase detection threshold.

The Jensen-Shannon divergence [8] is defined as:

$$\text{JSD}(P \| Q) = \frac{1}{2} D_{\text{KL}}(P \| M) + \frac{1}{2} D_{\text{KL}}(Q \| M) \quad (3)$$

where $M = \frac{1}{2}(P + Q)$ is the mixture distribution and D_{KL} is the Kullback-Leibler divergence:

$$D_{\text{KL}}(P \| Q) = \sum_x P(x) \ln \frac{P(x)}{Q(x)} \quad (4)$$

The token probability distribution $P(c)$ is computed by tokenizing the content (lowercased, stop-words removed, minimum 2 characters), computing term frequencies, and normalizing to a probability distribution.

1) **Preservation Strategy:** Phase detection identifies which thoughts to *preserve* during compaction:

- The first and last thoughts (trajectory endpoints).
- All phase boundary thoughts (maximum information content at topic transitions).
- The highest-confidence thought within each phase segment (representative of the segment’s conclusion).

Property 1 (Phase Boundary Information Maximum): At a phase boundary b_k , the mutual information between the content and the “topic” variable is locally maximal:

$$I(c_{b_k}; \text{topic}) \geq I(c_j; \text{topic}) \quad \forall j \in [b_{k-1} + 1, b_{k+1} - 1] \quad (5)$$

This follows from the JSD threshold: high JSD indicates that the token distributions before and after the boundary are maximally different, meaning the boundary thought carries the most information about the transition.

The mutual information is approximated using:

$$I(X; Y) \approx H(X) + H(Y) - H(X, Y) \quad (6)$$

where H is the Shannon entropy [7]:

$$H(P) = - \sum_x P(x) \log_2 P(x) \quad (7)$$

B. Orbit Compression

In the Lorenz system, trajectories orbit around two “eyes” of the strange attractor, tracing similar but not identical paths. Each orbit is qualitatively similar to the previous one. Applied to AI reasoning, an *orbit* is a repeating pattern where the model revisits the same conceptual territory from slightly different angles.

Definition 2 (Orbital Cycle): A subsequence $(t_s, t_{s+1}, \dots, t_{s+L-1})$ of length L is an *orbital cycle* if there exists a subsequent subsequence $(t_{s+L}, \dots, t_{s+2L-1})$ such that:

$$\frac{1}{L} \sum_{k=0}^{L-1} \text{sim}(t_{s+k}, t_{s+L+k}) \geq \tau_{\text{orbit}} \quad (8)$$

where $\text{sim}(t_i, t_j)$ is the TF-IDF cosine similarity between thoughts t_i and t_j , and $\tau_{\text{orbit}} = 0.5$ is the similarity threshold.

1) **Detection Algorithm:** The orbit detection algorithm operates in three phases:

- 1) **Similarity matrix construction:** Compute the $n \times n$ pairwise TF-IDF cosine similarity matrix $\mathbf{S}$ where $S_{ij} = \text{sim}(t_i, t_j)$.
- 2) **Greedy cycle search:** For cycle lengths $L \in [L_{\text{max}}, L_{\text{min}}]$ (longer first), scan starting positions and check for repetitions using (8). Mark claimed indices to prevent overlap.
- 3) **Compression:** For each detected cycle with r repetitions, keep the first occurrence and prune the remaining $r - 1$ copies.

The TF-IDF cosine similarity is computed as:

$$\text{sim}(t_i, t_j) = \frac{\mathbf{v}_i \cdot \mathbf{v}_j}{|\mathbf{v}_i| \cdot |\mathbf{v}_j|} \quad (9)$$

where $\mathbf{v}_i$ is the TF-IDF vector of thought t_i , with term frequency $\text{TF}(w, t) = \text{count}(w, t) / |t|$ and inverse document

Algorithm 1: Orbit Compression

Input : thoughts $\mathcal{T} = (t_1, \dots, t_n)$, threshold τ
Output : keep indices $\mathcal{K}$, prune indices $\mathcal{P}$

```

1  $\mathbf{S} \leftarrow \text{computeSimilarityMatrix}(\mathcal{T});$ 
2  $\text{claimed} \leftarrow \emptyset;$ 
3  $\text{orbits} \leftarrow [];$ 
4 for  $L \leftarrow \min(L_{\max}, \lfloor n/2 \rfloor)$  to  $L_{\min}$  do
5   for  $s \leftarrow 0$  to  $n - L \cdot R_{\min}$  do
6     if  $\exists k \in [s, s + L) : k \in \text{claimed}$  then
7       continue
8      $(r, \bar{S}) \leftarrow \text{detectRepetitions}(\mathbf{S}, s, L, \tau);$ 
9     if  $r \geq R_{\min}$  then
10       $\text{claimed} \leftarrow \text{claimed} \cup \{s, \dots, s + r \cdot L - 1\};$ 
11       $\text{orbits.push}(s, L, r, \bar{S});$ 
12  $\mathcal{P} \leftarrow \bigcup_{\text{orbit}} \text{orbit.pruneIndices};$ 
13  $\mathcal{K} \leftarrow \{1, \dots, n\} \setminus \mathcal{P};$ 

```

frequency $\text{IDF}(w) = \log(N/\text{df}(w))$, where N is the total number of thoughts and $\text{df}(w)$ is the number of thoughts containing word w .

Algorithm 1 formalizes the procedure.
 with constants $L_{\min} = 2$, $L_{\max} = 10$, $R_{\min} = 2$.

C. Perturbation Impact Analysis

The perturbation impact of an item t_k measures how much its removal changes the information landscape of the remaining context:

Definition 3 (Perturbation Impact): For a context set $\mathcal{C} = \{t_1, \dots, t_n\}$, the perturbation impact of item t_k is:

$$\Pi(t_k) = \text{JSD}(P(\mathcal{C}) \| P(\mathcal{C} \setminus \{t_k\})) \quad (10)$$

where $P(\mathcal{C})$ is the aggregate token distribution of all items in $\mathcal{C}$.

Items are ranked by $\Pi(t_k)$ in descending order. High-impact items are *causal anchors*—their removal would maximally perturb the system’s trajectory, analogous to the critical initial conditions in the Lorenz system.

Property 2 (Perturbation–Entropy Relationship): For a uniform context (all items have identical content), $\Pi(t_k) = 0$ for all k . For a maximally diverse context (each item introduces unique vocabulary), $\Pi(t_k)$ correlates with the information density:

$$\rho(t_k) = \frac{H(P(t_k))}{\log_2 |V(t_k)|} \quad (11)$$

where $|V(t_k)|$ is the vocabulary size of item t_k . Items with $\rho \approx 1$ have near-uniform token distributions and contribute maximally to the context’s information landscape.

D. Integrated Compaction Pipeline

The three algorithms are composed into a compaction pipeline:

- 1) **Phase detection:** Identify phase boundaries and mark boundary thoughts as *protected*.
- 2) **Orbit compression:** Within each phase segment, detect and collapse orbital cycles.

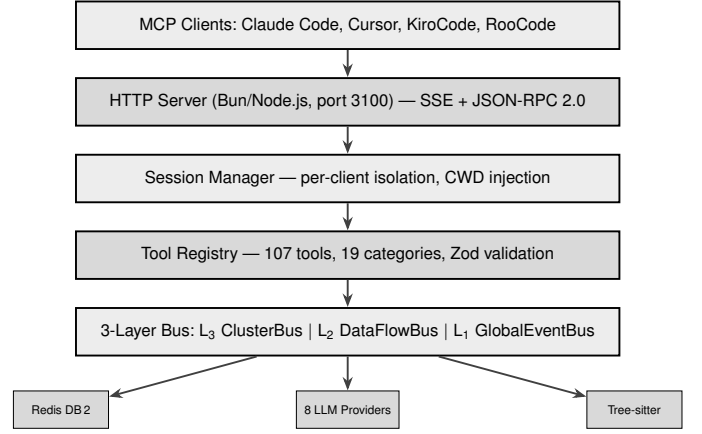


Figure 1. High-level system architecture. Version 3.0 reduces the tool registry from 147 to 107 tools by removing capabilities provided natively by IDE AI clients.

- 3) **Perturbation ranking:** Rank remaining unprotected thoughts by perturbation impact. Evict thoughts with lowest Π until the context budget is met.

The composition ensures that:

- Phase boundaries (maximum-information transitions) are never evicted.
- Redundant orbits are collapsed before perturbation ranking, preventing duplicate content from inflating impact scores.
- The final eviction uses information-theoretic ranking, not heuristic recency/frequency.

E. Additional CTC-Inspired Algorithms

The cognition module includes additional algorithms inspired by Closed Timelike Curves (CTC) from general relativity:

Causal DAG analysis. Constructs a directed acyclic graph of dependencies between cognition items and computes the causal core—the minimal set of items needed to reconstruct all others—using transitive reduction.

Fixed-point detection. Iteratively compacts a thinking chain until convergence, defined as:

$$|C_{i+1}| \geq |C_i| \cdot (1 - \epsilon), \quad \epsilon = 0.05 \quad (12)$$

This mirrors the Novikov self-consistency principle: the compacted result must be a fixed point that regenerates itself under further compaction.

Gravitational TTL. Assigns time-to-live based on item “mass” (information density $\times$ link count), analogous to gravitational time dilation:

$$\text{TTL}(t_k) = \text{TTL}_{\text{base}} \cdot (1 + \alpha \cdot m(t_k)) \quad (13)$$

where $m(t_k)$ is the normalized mass and $\alpha = 0.5$ is the amplification factor. High-mass items live longer.

IV. SYSTEM ARCHITECTURE

Fig. 1 shows the high-level system architecture. The server is organized into five principal layers: transport, session management, tool registry, bus architecture, and shared infrastructure.

Table I
HTTP ENDPOINTS

Method	Path	Purpose
GET	/sse	SSE notification stream
POST	/message	JSON-RPC tool invocations
GET	/stream/:id	Per-request execution stream
GET	/health	Health check and uptime
GET	/tools	Tool listing with schemas
GET	/mcp	Streamable HTTP endpoint

A. Transport Layer

The server implements the MCP Streamable HTTP transport, exposing the endpoints listed in Table I. Communication follows JSON-RPC 2.0 with the MCP method extensions. Server-Sent Events (SSE) [23] provide the server-to-client notification channel.

B. Tool Registry

The tool registry manages 107 tools through a unified `UnifiedTool` interface. Each tool defines its input schema using `Zod` [12], which is converted to JSON Schema for the MCP protocol. Lazy loading ensures that tool implementations are instantiated on first invocation. The availability checker reports tool health with soft and force modes, suggesting fallback alternatives when dependencies are unavailable.

C. Tool Removal Rationale (v3.0)

Version 3.0 removes 39 tools across 8 categories: file operations (9), git (9), line editing (4), symbol editing (3), code navigation (2), code modification (5), AI specialized (2), and project management (5). These tools duplicated capabilities that modern AI IDEs provide natively:

- **Claude Code:** built-in Read, Write, Edit, Bash, Glob, Grep tools.
- **Cursor:** native file editing, terminal, and code navigation.
- **KiroCode/RooCode:** equivalent built-in tool sets.

The internal utility modules (`utils/file-utils.ts`, `utils/git-utils.ts`) are preserved, as they are used by remaining tools (e.g., `write-tests` uses file reading, `smart-handoff` uses git status).

V. PROTOCOL LAYER ARCHITECTURE

The communication infrastructure consists of three layers connected by anti-amplification bridges. Fig. 2 illustrates the layered design.

A. L_3 : ClusterBus

The ClusterBus enables inter-cluster communication over Redis Pub/Sub [9] with 18 signal types organized into four namespaces plus CI/CD. Four send modes are supported: unicast, broadcast, routed (via `MetaRouter` with tag-based boolean logic), and multicast (fan-out to multiple targets).

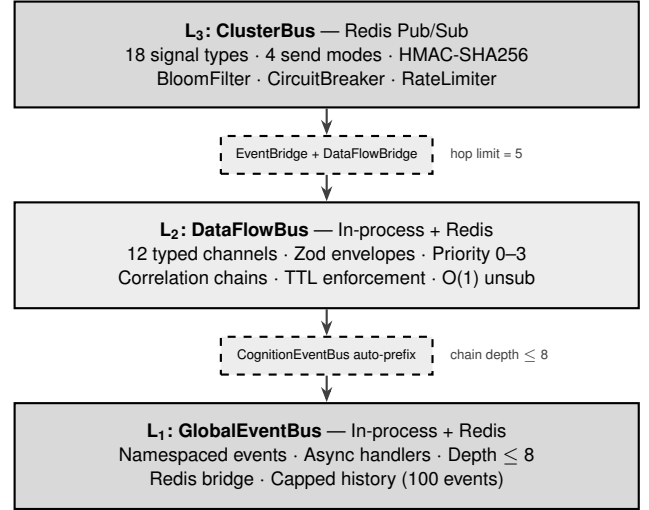


Figure 2. Three-layer bus architecture with anti-amplification bridges.

1) *Dual-Layer Deduplication:* A bounded set of 2,000 recent signal IDs provides exact deduplication. Overflow is handled by a bloom filter [20] configured for 20,000 items at 0.1% false-positive rate:

$$m = -\frac{n \ln p}{(\ln 2)^2}, \quad k = \frac{m}{n} \ln 2 \quad (14)$$

2) *HMAC Authentication:* Signals are authenticated using HMAC-SHA256 with constant-time comparison to prevent timing side-channel attacks.

3) *SignalFlowController:* Governs throughput through rate limiting, circuit breaking [22] (closed → open → half-open automaton), priority filtering, and backpressure control.

B. L_2 : DataFlowBus

12 typed channels with `Zod`-validated envelopes, correlation tracking, priority 0–3, and TTL enforcement. Messages are persisted to Redis under `mcp:dataflow:{channel}`.

C. L_1 : GlobalEventBus

In-process event emitter with namespaced events (`module:category:action`), async dispatch with chain depth limit of 8, and Redis bridge for cross-session propagation.

D. Anti-Amplification Bridges

Four guards prevent feedback loops: hop counter (max 5), source prefix filtering, per-bridge dedup set, and chain depth limiting.

Invariant 1 (Signal Termination): For any signal s , the total copies across all layers is bounded by:

$$|C(s)| \leq H_{\max} \cdot D_{\max} = 5 \times 8 = 40 \quad (15)$$

In practice, deduplication reduces this to $|C(s)| \leq 3$.

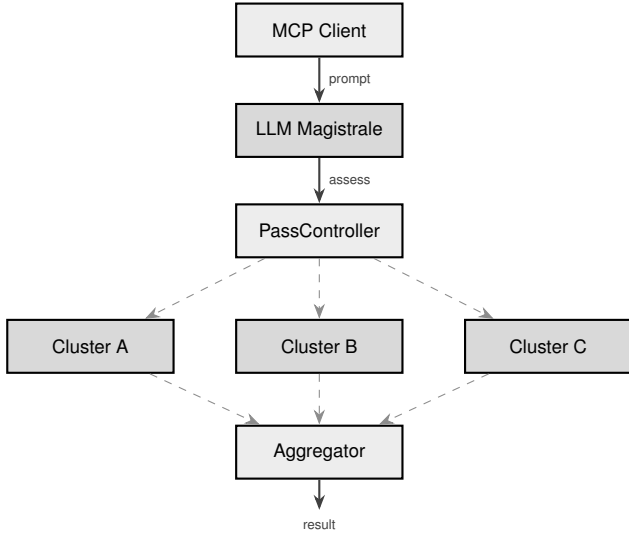


Figure 3. LLM Magistrale dispatch flow. Solid arrows: control flow; dashed: async ClusterBus signals.

VI. LLM MAGISTRALE

The LLM Magistrale orchestrates distributed prompt execution across multiple clusters. Fig. 3 illustrates the dispatch flow.

A. Aggregation Strategies

Four strategies are supported: *first-wins* (latency-optimized), *best-of-n* with composite scoring, *consensus* (weighted Jaccard similarity, threshold 0.6), and *fallback-chain* (sequential priority). The composite score is:

$$S = w_q \cdot Q + w_d \cdot D + w_t \cdot T - w_l \cdot L \quad (16)$$

with default weights $w_q = 0.45$, $w_d = 0.25$, $w_t = 0.15$, $w_l = 0.15$.

B. PassController

Three strategies govern multi-pass execution: *min-passes* (LLM self-assesses), *quality-target* (iterate until $Q_i \geq Q_{\text{target}}$), and *fixed* (exact N passes). Budget capping limits `minPasses` to `maxPasses`.

VII. COGNITION SUBSYSTEM

The cognition layer provides persistent AI self-awareness through eight primary stores, two auxiliary analyzers, and a cross-linking mechanism. Fig. 4 shows the store interconnections.

The eight primary stores are: *DecisionJournal* (structured decisions with outcome tracking), *ConfidenceTracker* (calibrated scores with ECE monitoring), *IntentTracker* (hierarchical goals with TF-IDF drift detection), *ErrorPatternDB* (cross-session error database), *SelfCritiqueStore* (post-task reflections), *MentalModelStore* (component-relationship architecture maps), *HandoffStore* (auto-enriched session snapshots), and *ContextBudgetManager* (token estimation with relevance scoring).

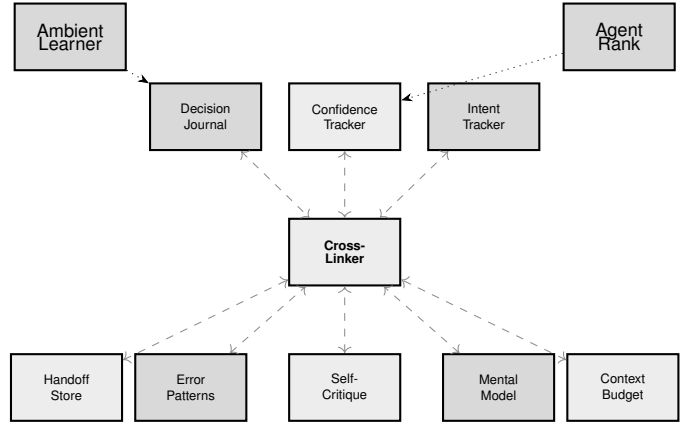


Figure 4. Cognition stores interconnected via CognitionCrossLinker.

Agent ranking uses a composite score with five tiers (bronze through diamond):

$$\text{Score} = 0.4 \cdot S_r + 0.3 \cdot C_t + 0.2 \cdot A_s + 0.1 \cdot C_o \quad (17)$$

with exponential temporal decay $\text{Score}(t) = \text{Score}_0 \cdot 0.95^{\Delta t / \tau}$.

Nine reactive event handlers create feedback loops between stores (e.g., `decision:recorded` auto-creates confidence records, `intent:drifted` triggers self-critique).

VIII. TOOL CATEGORIES

Table II enumerates all 19 tool categories with counts and representative tools.

IX. MULTI-AGENT COORDINATION

KemdiCode supports multi-agent workflows through four mechanisms:

Agent Registry. Agents register with role, capabilities, and status. Discovery via `agent-list`; monitoring via `monitor` with five views (overview, agents, tasks, hierarchy, activity).

Message Passing. `agent-inject` sends context to specific agents via Redis Pub/Sub. `agent-alert` broadcasts to all agents. `shared-thoughts` provides a collective knowledge base.

Kanban System. Hierarchical task management: workspaces contain boards, boards contain tasks with subtask support. Role-based membership (owner, admin, member, viewer) controls access.

Task Intelligence. `task-cluster` uses LLM-driven semantic clustering. `task-complexity` provides AI-powered difficulty estimation.

X. SECURITY

Security mechanisms include: Zod schema validation on all inputs, path traversal protection via `validatePathSafe()`, Redis JSON parsing with prototype pollution protection, HMAC-SHA256 cluster signal authentication with constant-time comparison, Shamir's Secret Sharing [10] for distributed secret management, and ReDoS prevention through regex pattern length caps.

Table II
TOOL CATEGORIES (107 TOTAL)

Category	#	Key Tools
Core AI	6	ask-ai, plan, build
Code Intelligence	4	find-definition, semantic-search
Multi-LLM	3	multi-prompt, consensus
Cognition	8	decision-journal, confidence
Multi-Agent	10	agent-init, agent-rank
Context	3	shared-thoughts, feedback
Kanban Tasks	13	task-create, task-cluster
Kanban Workspaces	5	workspace-create, -join
Kanban Boards	7	board-create, board-workflow
Project Memory	8	write-memory, checkpoint-save
Recursive	4	invoke-tool, orchestrate
Session	6	session-recover, session-create
Thinking	1	thinking-chain
Knowledge Graph	4	graph-query, loci-recall
Cluster Bus	8	bus-status, -magistrale
MPC Security	4	mpc-split, mpc-reconstruct
RL Learning	2	rl-reward-stats
MCP Client	3	client-sampling, client-elicite
System	8	env-info, tool-health, ping

XI. LLM PROVIDER INTEGRATION

Eight providers with unified provider: `model:thinking` syntax: Anthropic (a), OpenAI (o), Gemini (g) with native SDKs, and Groq (q), DeepSeek (d), Ollama (l), OpenRouter (r), Perplexity (p) via OpenAI-compatible adapters. Three-tier routing (primary/research/fallback), lazy initialization, hot-reload, and structured output via `generateObject()` with Zod schemas and `jsonrepair`.

XII. EVALUATION

A. Test Suite

694 passing tests across 29 test files covering all tool categories, bus layers, cognition stores, context compaction algorithms, and edge cases. Tests are executed via Vitest [13] with the V8 coverage provider.

B. Client Compatibility

Verified with four MCP clients: Claude Code (primary), Cursor, KiroCode, and RooCode.

C. Infrastructure

Dual-runtime support (Bun and Node.js). Tree-sitter [11] WASM parsers for 19 languages. All persistent state uses Redis DB 2 with 10 key namespaces.

D. Tool Reduction Impact

The removal of 39 tools reduces the server's surface area by 27%. The remaining 107 tools have zero overlap with native IDE capabilities. Startup time improves proportionally due to fewer lazy-loadable modules. No functional regression: all removed capabilities are available natively in supported MCP clients.

XIII. CONCLUSION

KemdiCode MCP Server v3.0.0 demonstrates two key contributions. First, that chaos theory and dynamical systems provide a rigorous framework for AI context compaction: Poincaré section phase detection identifies information-dense topic transitions, Lorenz attractor orbit compression eliminates redundant reasoning cycles, and perturbation impact analysis quantifies each thought's structural importance. Together, these algorithms replace heuristic recency-based eviction with information-theoretic compression.

Second, that MCP tool servers should focus on capabilities that AI IDEs cannot provide natively. By removing 39 redundant tools, v3.0 achieves a 27% reduction in tool count while improving focus: the remaining 107 tools cover cognition, multi-agent coordination, persistent memory, distributed LLM orchestration, and context compaction—domains where protocol-level tooling is essential.

The three-layer bus architecture, eight-store cognition subsystem, and multi-provider LLM magistrale remain core infrastructure. Future work includes embedding-based similarity (replacing TF-IDF) for higher-fidelity orbit detection, cross-project knowledge graph learning, and reinforcement learning for autonomous compaction threshold tuning.

REFERENCES

- [1] Anthropic, "Introducing the Model Context Protocol," Anthropic Blog, Nov. 2024. [Online]. Available: <https://www.anthropic.com/news/model-context-protocol>
- [2] Model Context Protocol Authors, "Model Context Protocol Specification, version 2025-11-25," 2025. [Online]. Available: <https://modelcontextprotocol.io/specification/2025-11-25>
- [3] JSON-RPC Working Group, "JSON-RPC 2.0 Specification," 2010. [Online]. Available: <https://www.jsonrpc.org/specification>
- [4] Microsoft, Red Hat, and Codenvy, "Language Server Protocol Specification," 2016. [Online]. Available: <https://microsoft.github.io/language-server-protocol/>
- [5] E. N. Lorenz, "Deterministic nonperiodic flow," *J. Atmospheric Sci.*, vol. 20, no. 2, pp. 130–141, Mar. 1963. doi:10.1175/1520-0469(1963)020<0130:DNF>2.0.CO;2
- [6] H. Poincaré, "Sur le problème des trois corps et les équations de la dynamique," *Acta Math.*, vol. 13, pp. 1–270, 1890.
- [7] C. E. Shannon, "A mathematical theory of communication," *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, Jul. 1948. doi:10.1002/j.1538-7305.1948.tb01338.x
- [8] J. Lin, "Divergence measures based on the Shannon entropy," *IEEE Trans. Inform. Theory*, vol. 37, no. 1, pp. 145–151, Jan. 1991. doi:10.1109/18.61115
- [9] S. Sanfilippo, "Redis: An in-memory data structure store," 2009–2026. [Online]. Available: <https://redis.io/>
- [10] A. Shamir, "How to share a secret," *Commun. ACM*, vol. 22, no. 11, pp. 612–613, Nov. 1979. doi:10.1145/359168.359176
- [11] M. Brunsfeld *et al.*, "Tree-sitter: An incremental parsing system for programming tools," 2018–2026. [Online]. Available: <https://tree-sitter.github.io/>
- [12] C. McDonnell, "Zod: TypeScript-first schema validation with static type inference," 2020–2026. [Online]. Available: <https://zod.dev/>

- [13] A. Fu *et al.*, “Vitest: A Vite-native unit testing framework,” 2022–2026. [Online]. Available: <https://vitest.dev/>
- [14] J. Li, Q. Wang *et al.*, “A survey on LLM-based multi-agent systems: Workflow, infrastructure, and challenges,” *Vicinagearth*, Oct. 2024. doi:10.1007/s44336-024-00009-2
- [15] T. Guo, X. Chen *et al.*, “Large language model based multi-agents: A survey of progress and challenges,” in *Proc. 33rd Int. Joint Conf. Artif. Intell. (IJCAI)*, 2024.
- [16] Q. Wu, G. Bansal *et al.*, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” *arXiv:2308.08155*, 2023.
- [17] S. Hong, X. Zhuge *et al.*, “MetaGPT: Meta programming for multi-agent collaborative framework,” *arXiv:2308.00352*, 2023.
- [18] J. Wu *et al.*, “Recursively summarizing books with human feedback,” *arXiv:2109.10862*, 2021.
- [19] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” *Advances in Neural Inform. Process. Syst.*, vol. 33, pp. 9459–9474, 2020.
- [20] B. H. Bloom, “Space/time trade-offs in hash coding with allowable errors,” *Commun. ACM*, vol. 13, no. 7, pp. 422–426, Jul. 1970. doi:10.1145/362686.362692
- [21] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” *Commun. ACM*, vol. 21, no. 7, pp. 558–565, Jul. 1978. doi:10.1145/359545.359563
- [22] M. T. Nygard, *Release It! Design and Deploy Production-Ready Software*. Raleigh, NC, USA: Pragmatic Bookshelf, 2007.
- [23] I. Hickson, “Server-Sent Events,” W3C Recommendation, Feb. 2015. [Online]. Available: <https://www.w3.org/TR/eventsource/>